



Department of Electrical and Information Engineering
University of Ruhuna

Project Report
High Performance Computing
EC7207 / EE7218

Parallel Pearson Correlation
Based Recommendation System

Serial · OpenMP · POSIX Threads · MPI · Hybrid MPI+OpenMP · Hybrid
MPI+Pthreads · CUDA

Group 11

EG/2021/4417 Ashfaq M.R.M.

EG/2021/4419 Athanayaka K.A.L.G.

EG/2021/4424 Balasooriya J.M.

Submission date: May 23, 2026

Contents

1	Introduction and Motivation	1
2	Algorithm Design and Parallel Programming Concepts	1
2.1	Algorithm Pipeline (Five Phases)	1
2.2	OpenMP: Shared Memory Fork Join Parallelism	2
2.3	Pthreads: Manual POSIX Thread Management	3
2.4	MPI: Distributed Memory Parallelism	4
2.5	CUDA: GPU Massively Parallel Computing	4
2.6	Hybrid MPI+OpenMP: Two Level Parallelism	6
2.7	Hybrid MPI+Pthreads: Two Level Parallelism, POSIX Inner Layer	6
3	Experimental Setup	7
4	Correctness Analysis: Parallel vs. Serial	8
5	Timing Measurements and Performance Analysis	9
5.1	Performance Metric Definitions	9
5.2	Serial Baseline	9
5.3	OpenMP: Varying Thread Count	9
5.4	Pthreads: Varying Thread Count	9
5.5	MPI: Varying Process Count	10
5.6	Hybrid MPI+OpenMP: Fixed 8 Total Workers	10
5.7	Hybrid MPI+Pthreads: Fixed 8 Total Workers	10
5.8	CUDA: GPU Benchmark	10
5.9	Scalability Across Problem Sizes	11
6	Performance Discussion	12
6.1	Head to Head Comparison at 8 Workers	12
6.2	OpenMP vs. Pthreads	12
6.3	MPI Scaling	12
6.4	Hybrid MPI+OpenMP: Inversion on a Single Node	12
6.5	Hybrid MPI+OpenMP vs. MPI+Pthreads	13
6.6	Amdahl's Law	13
7	Conclusions	13
	References	14

1 Introduction and Motivation

Modern recommendation systems generate personalised suggestions by comparing a target user's preferences with the preferences of many other users. This project focuses on that problem: as the number of users and items grows, the similarity calculation becomes too slow for a simple serial program. Pearson correlation is used because it measures how closely two users' rating patterns move together. After the system finds similar users, it predicts an unseen rating by combining the neighbours' ratings with weights based on their similarity.

The main cost appears during user similarity computation. For the benchmark data used in this report, there are 1,000 users and 1,000 items. Each user must be compared with all other users, and each comparison may scan many item ratings to compute one Pearson similarity value. That creates hundreds of thousands of user pair comparisons and roughly hundreds of millions of rating checks before prediction begins. This is the exact situation described in the proposal: large similarity computations are useful for recommendation systems, but they become slow as the data size increases. Therefore the Pearson similarity phase is the main bottleneck and the main target for parallelisation.

This report documents a **Pearson correlation based recommender** implemented in seven versions: serial, OpenMP, POSIX threads, MPI, CUDA, hybrid MPI+OpenMP, and hybrid MPI+Pthreads. All versions use the same synthetic ratings matrix (1,000 users $\times$ 1,000 items, SEED=42, 70% sparse, TOP K = 20), so the report compares only the effect of the parallel programming model. The evaluation checks two questions: whether each parallel version preserves the serial result, and how much runtime improves as workers increase from 1 to 8. Every parallel version reproduces the serial result exactly. For this implementation-correctness evaluation, MAE and RMSE are measured as the average error of each implementation's prediction array against the serial prediction array. Therefore the serial baseline and every parallel version report MAE = 0.0000 and RMSE = 0.0000; the similarity-matrix checksum is bit-identical at 942.387323, confirming race free partitioning. CUDA is fastest at 38.78 $\times$ over serial; OpenMP leads the CPU versions at 6.26 $\times$ on eight threads.

Platform. Pop!_OS 24.04 LTS · GCC 13.3.0 · Open MPI 4.1.6 · Intel Core i7 11800H (8 physical / 16 logical cores) · NVIDIA GeForce RTX 3050 Laptop GPU.

2 Algorithm Design and Parallel Programming Concepts

2.1 Algorithm Pipeline (Five Phases)

Every version executes the same five sequential phases summarised in Table 1. Phases 3 and 4 dominate runtime and are the main parallelisation targets; Phase 2 is also row parallel in the parallel versions. Data generation stays serial in all versions because C's `rand()` is not thread safe, and running the same `srand(42)` sequence everywhere guarantees a byte identical input matrix without any communication.

The two heavy phases use the following formulas. Pearson similarity (Phase 3):

$$\text{sim}(u, v) = \frac{\sum_i (R_{ui} - \mu_u)(R_{vi} - \mu_v)}{\sqrt{\sum_i (R_{ui} - \mu_u)^2} \sqrt{\sum_i (R_{vi} - \mu_v)^2}}$$

and the TOP K weighted prediction (Phase 4):

$$\hat{r}_{ui} = \mu_u + \frac{\sum_{k \in N_K(u)} \text{sim}(u, k) (R_{ki} - \mu_k)}{\sum_{k \in N_K(u)} \text{sim}(u, k)}.$$

Both are *embarrassingly parallel at the row level*: the work for one user row is independent of every other, which is what makes the same algorithm map cleanly onto all five parallel models.

Phase	Name	Description	Complexity
1	Data Generation	Synthetic sparse matrix; 10% test split	$O(N \times M)$
2	User Means	Mean for each user μ_u	$O(N \times M)$
3	Similarity	Pearson for all (u, v) pairs, the bottleneck	$O(N^2 \times M)$
4	Predictions	TOP K weighted prediction	$O(N^2 \times M)$
5	Correctness Evaluation	MAE/RMSE against serial predictions	$O(T)$

Table 1: Algorithm phases and computational complexity.

Figure 1 shows the pipeline and marks which phases each technology parallelises; the row wise decomposition that keeps every version race free is then detailed in the per technology diagrams that follow.

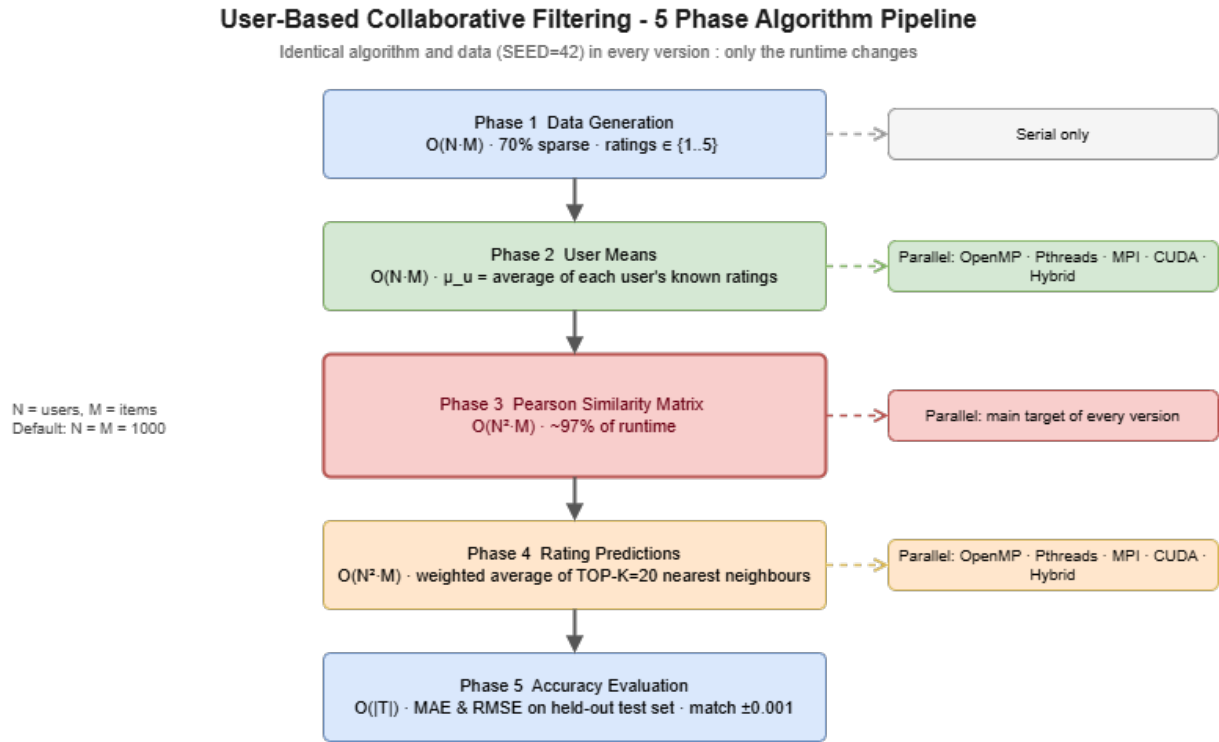


Figure 1: The five phase recommender pipeline, shared by every implementation. The same algorithm and data (SEED=42) are used throughout; only the runtime changes. Phase 3 (Pearson similarity) is the $O(N^2M)$ bottleneck; Phases 3–4 are the main targets of all five parallel models, Phase 2 is also row parallel in the parallel implementations, and data generation stays serial.

2.2 OpenMP: Shared Memory Fork Join Parallelism

OpenMP exploits shared memory parallelism through compiler directives: a master thread forks into T workers that share one address space, then rejoins at an implicit barrier (Figure 2). The decisive design choice is in Phase 3, where the outer loop carries `#pragma omp parallel for schedule(dynamic,4)`. Dynamic scheduling is essential because the triangular similarity loop is badly imbalanced; the first row performs $N-1 = 999$ Pearson calls while the last performs only one. Dynamic scheduling lets idle threads pull four row chunks from a shared work queue, so no core sits idle.

The version stays lock free without any mutex: each thread owns an exclusive set of outer loop

rows u and writes $\text{SIM}(u,v)$ only for those rows, so no two threads ever touch the same cell. In Phase 5 the `reduction(+:err)` clause gives each thread a private accumulator that OpenMP sums at the barrier, and the same pattern (`reduction(+:sq)`) accumulates the squared prediction difference for RMSE.

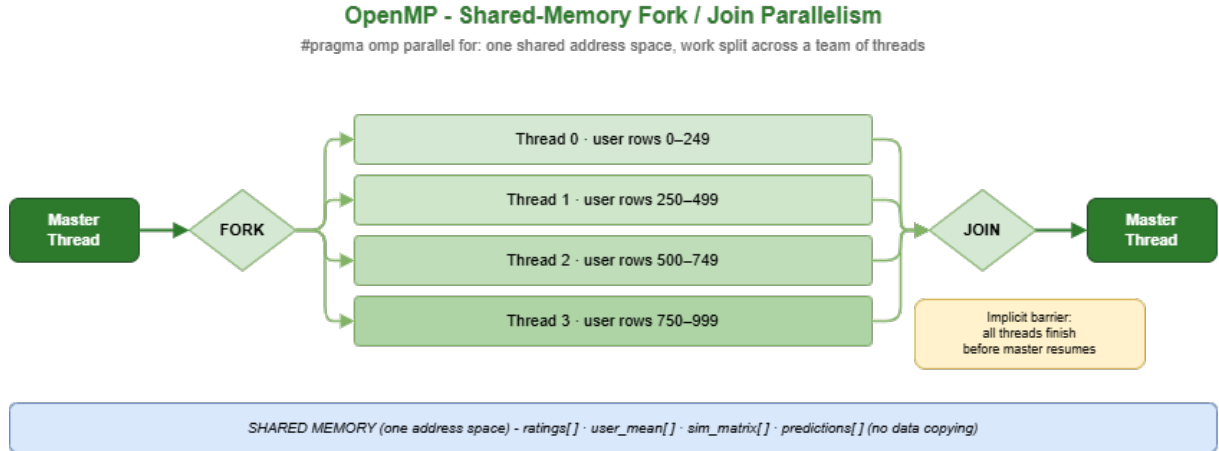


Figure 2: OpenMP fork/join model. The master thread forks a team of T workers over one shared address space (`ratings`, `sim_matrix`, ...), each owning a disjoint slice of user rows, then rejoins at an implicit barrier. The side notes show the three race free techniques used: `schedule(dynamic,4)` for the triangular loop, disjoint row writes, and a thread private `nbrs[]` buffer with `reduction(+:err)`.

2.3 Pthreads: Manual POSIX Thread Management

POSIX threads require the programmer to create threads, assign work, and synchronise explicitly (Figure 3). A `run_parallel(fn, nthreads)` harness is invoked at the start of Phases 2, 3, and 4: it calls `pthread_create()` to spawn T threads, each receiving a `ThreadArgs` struct with its `tid` and the total `nthreads`, lets every thread derive its exclusive row range `start = tid × ⌈N/T⌉`, `end = min(start + ⌈N/T⌉, N)`, and then uses `pthread_join()` as a phase barrier.

The key difference from OpenMP is twofold: threads are created and joined three separate times (once per phase), paying OS thread creation overhead three times per run, and the static ceiling division row assignment cannot compensate for the triangular loop imbalance. Both effects explain the slightly lower efficiency observed against OpenMP in Section 5.

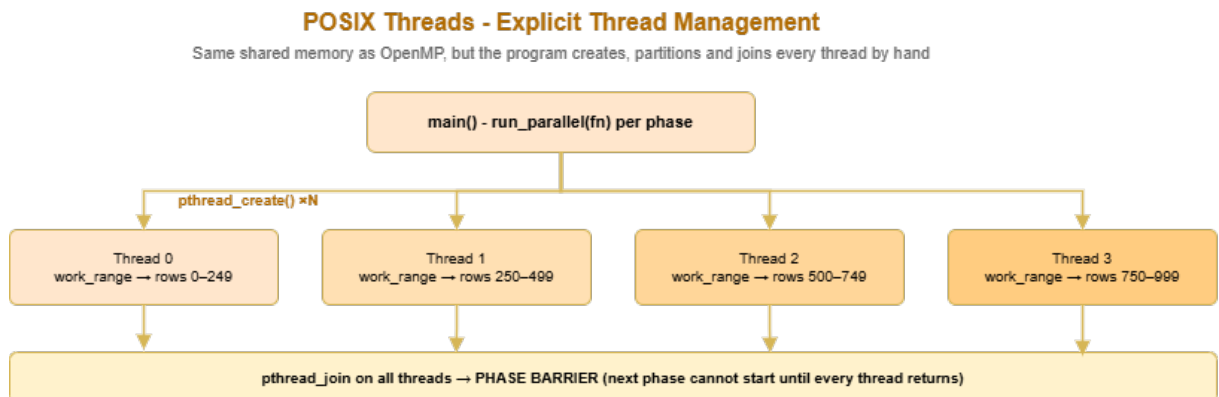


Figure 3: Manual POSIX thread management. `main()` calls `run_parallel` once per parallel phase: `pthread_create` spawns T threads, each deriving its own `ceil` division row range from a `ThreadArgs` struct, and `pthread_join` on all threads acts as the phase barrier. Disjoint rows plus a private `nbrs[]` buffer make it lock free.

2.4 MPI: Distributed Memory Parallelism

Each MPI rank is an independent process with its own private address space (Figure 4). The $N = 1,000$ users are divided evenly: rank r owns rows $[r \cdot N/P, (r+1) \cdot N/P)$. Because the ranks cannot share memory, data has to be exchanged explicitly through collective operations. Concretely: Phase 1 needs no communication (all ranks call `srand(42)` independently); Phase 2 computes a local `user_mean[]` slice and gathers it with `MPI_Allgatherv`; Phase 3 fills the rank's rows of the similarity matrix and rebuilds the full matrix on every rank, again with `MPI_Allgatherv`; Phase 4 predicts the rank's own users with no communication; and Phase 5 sums the local serial-vs-parallel prediction error to rank 0 via `MPI_Reduce(SUM)` for the global MAE/RMSE.

The `MPI_Allgatherv` that rebuilds the $N \times N$ similarity matrix transfers ≈ 4 MB at $N = 1,000$. This is the cost that limits single node efficiency, but it is also exactly the mechanism that makes the code work unchanged across multiple physical nodes, where shared memory is not an option.

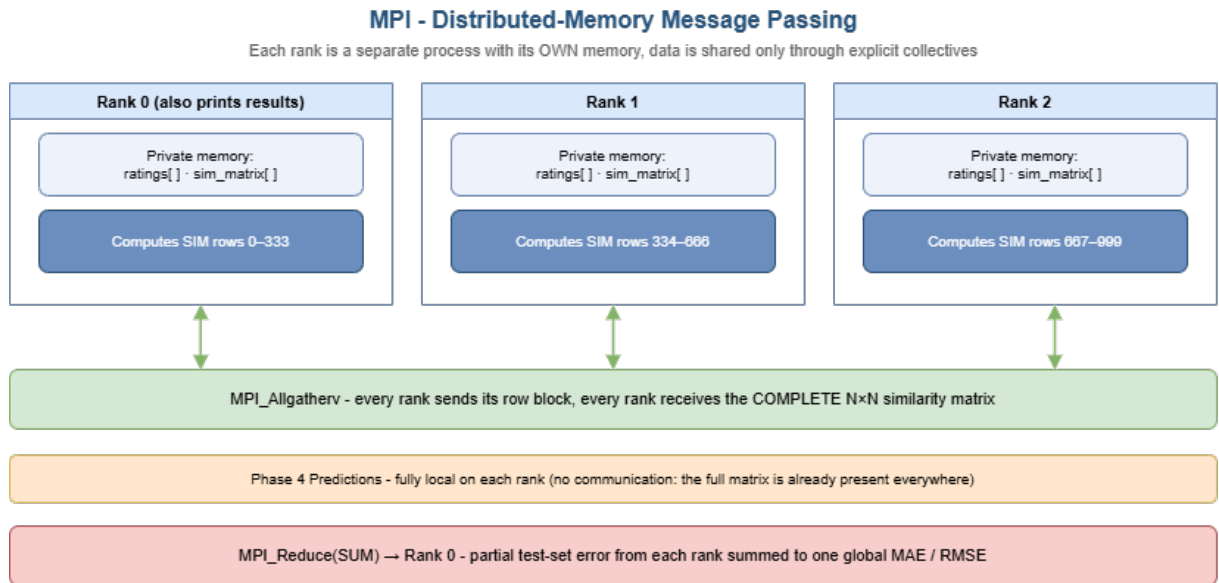


Figure 4: MPI distributed memory layout. Each rank is a separate process with a private copy of the matrices, computing the similarity rows it owns. A single `MPI_Allgatherv` then gives every rank the complete $N \times N$ matrix (so predictions need no communication), and `MPI_Reduce` sums the partial serial-vs-parallel prediction error to rank 0 for the global MAE/RMSE.

2.5 CUDA: GPU Massively Parallel Computing

The CUDA version (`cuda/cuda_recommender.cu`) is fully implemented and benchmarked on an NVIDIA GeForce RTX 3050 Laptop GPU (Ampere, compute capability 8.6, 16 SMs); measured results appear in Section 5.8. It maps the embarrassingly parallel structure of the problem onto one GPU thread per output element, using three kernels: `kernel_user_means` (one thread per user, launched as $\ll(N+255)/256, 256\gg$), `kernel_similarity` (one thread per (u, v) pair on a 2D grid of 16×16 thread blocks), and `kernel_predictions` (one thread per (u, i) cell on a 2D users $\times$ items grid).

The similarity kernel alone launches a grid with $63 \times 63 \times 256 = 1,016,064$ logical threads (Figure 5). Because device code cannot call `qsort`, each prediction thread keeps its TOP K neighbours in a small fixed size register buffer instead. Timing uses `cudaEventRecord` so that the reported kernel time excludes the `cudaMemcpy` transfer overhead.

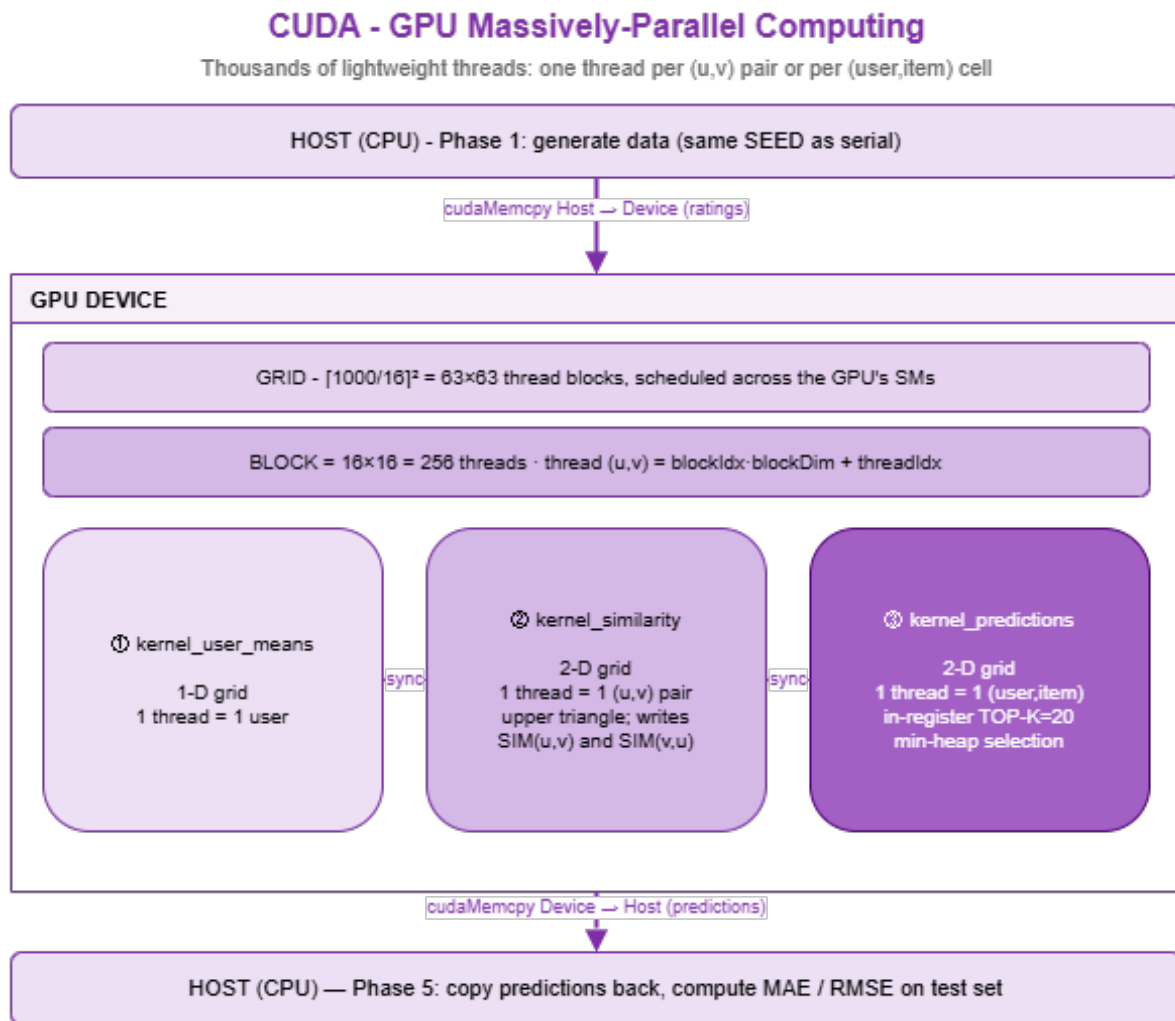


Figure 5: CUDA GPU model. After a Host→Device copy of the ratings, three kernels run in sequence on a 2D grid of 16×16 thread blocks, with one thread per (u,v) pair / $(user,item)$ cell, before the predictions are copied back. The left column shows the GPU memory hierarchy (registers → shared → global → host). Benchmarked on an NVIDIA RTX 3050 (compute capability 8.6).

2.6 Hybrid MPI+OpenMP: Two Level Parallelism

The hybrid version combines coarse grain MPI (across nodes) with fine grain OpenMP (within a node), as shown in Figure 6. With P MPI ranks and T OpenMP threads per rank, the total worker count is $P \times T$ while only P messages are sent per collective call. The `MPI_THREAD_FUNNELED` mode (selected via `MPI_Init_thread`) guarantees that every MPI call is made from the main thread only, after the OpenMP parallel regions have closed. Four layouts are tested at a fixed eight total workers: 2×4 (fewer MPI messages, more shared memory parallelism), 4×2 (balanced communication vs. threading overhead), 8×1 (equivalent to pure MPI), and 1×8 (pure OpenMP wrapped with MPI overhead).

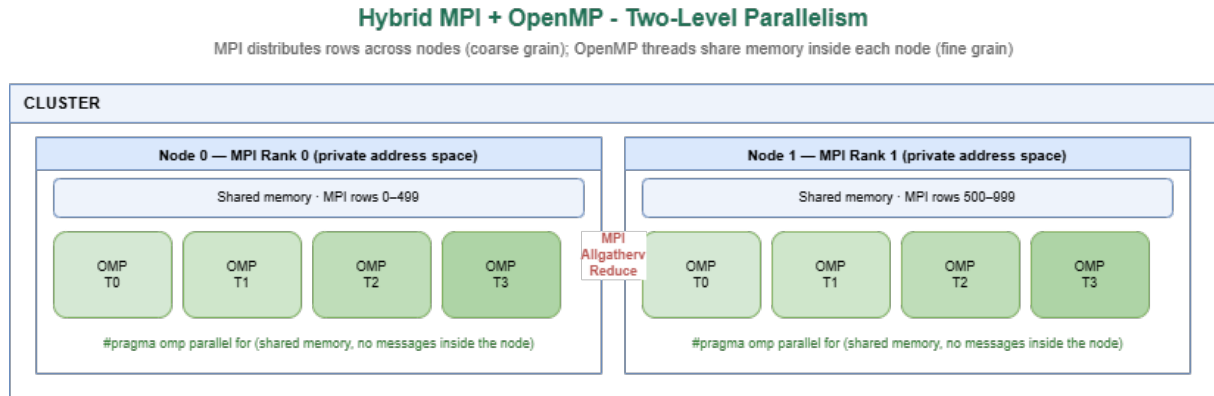


Figure 6: Hybrid two level parallelism. Level 1 (MPI) partitions user rows across P ranks on separate nodes; level 2 (OpenMP) spawns T threads over each rank’s rows in shared memory, for $P \times T$ total workers. `MPI_Allgatherv` / `MPI_Reduce` link the ranks, and `MPI_THREAD_FUNNELED` keeps all MPI calls on the main thread.

2.7 Hybrid MPI+Pthreads: Two Level Parallelism, POSIX Inner Layer

A second hybrid variant keeps the outer MPI layer unchanged and replaces the inner OpenMP layer with raw POSIX threads (Figure 7). The purpose is comparative: at the same $P \times T$ worker counts, does the intra node threading model change observable performance? The outer structure is byte identical to Section 2.6, with the same `get_range` row partition, same `MPI_THREAD_FUNNELED` contract, same `MPI_Allgatherv` after Phases 2 and 3, and the same `MPI_Reduce` for the global prediction error. Only the inner loop differs: a `run_parallel(fn, threads, args, start, end)` harness spawns T pthreads per phase, each carrying its rank’s row window in its `ThreadArgs` and deriving its sub range via the same `work_range` ceiling division used by the pure pthreads version. Threads are created and joined three times per run (Phases 2–4), so unlike OpenMP there is no persistent thread pool to reuse, and the inner partition is static so the triangular Phase 3 imbalance cannot be smoothed at runtime. The same four $P \times T$ layouts are tested as in the MPI+OpenMP hybrid (2×4 , 4×2 , 8×1 , 1×8 , all eight total workers), so the two variants can be compared head to head; at 8×1 this reduces to pure MPI (one pthread per rank), and at 1×8 to pure pthreads wrapped with MPI overhead.

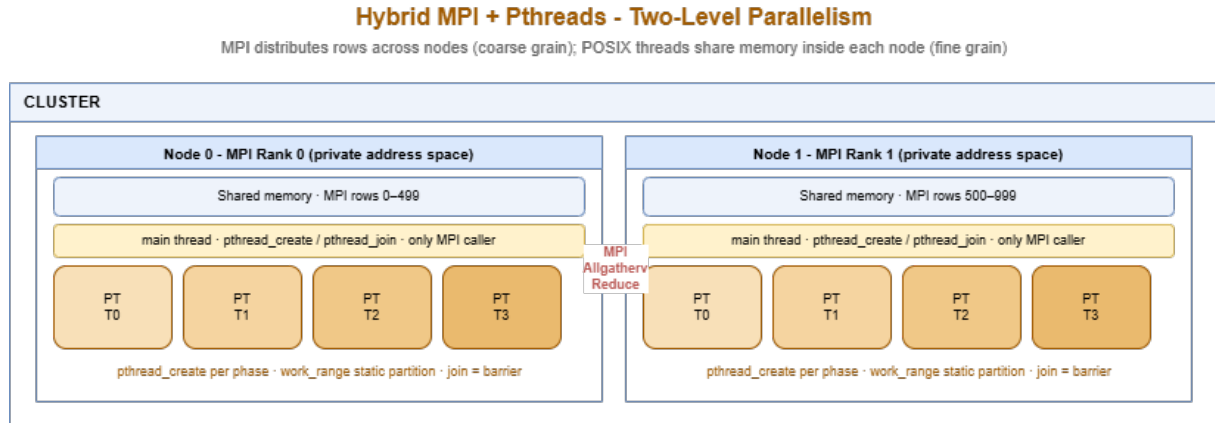


Figure 7: Hybrid MPI+Pthreads two level parallelism. Level 1 (MPI) is unchanged from Figure 6; level 2 replaces the OpenMP thread team with pthreads that are `pthread_create`'d and `pthread_join`'d once per parallel phase. `MPI_THREAD_FUNNELED` still applies: collectives run on the main thread only, after the join barrier.

3 Experimental Setup

All versions are benchmarked on a single Linux laptop: Intel Core i7 11800H (8 physical / 16 logical cores) with an NVIDIA GeForce RTX 3050 Laptop GPU for the CUDA version. The fixed parameters held constant across every run are listed in Table 2.

Parameter	Value
Users (N)	1 000
Items (M)	1 000
Sparsity	70%
Test split	10% (29 866 held out ratings)
TOP K neighbours	20
Random seed	42
Benchmark thread/process counts	1, 2, 4, 8
Timing method	<code>clock_gettime</code> / <code>omp_get_wtime</code> / <code>MPI_Wtime</code> / <code>cudaEvent</code> (GPU)
Platform	Pop!_OS 24.04 LTS, GCC 13.3.0, Open MPI 4.1.6, CUDA 12.0
CPU / GPU	Intel Core i7 11800H (8 phys / 16 logical) / NVIDIA RTX 3050 Laptop GPU

Table 2: Fixed benchmark parameters used across all versions.

Fairness controls. The comparison is kept fair by construction:

- The same `SEED=42` gives every version a byte identical ratings matrix and test split, since Phase 1 runs the same serial `rand()` sequence everywhere.
- The same fixed parameters ($N = M = 1,000$, 70% sparsity, TOP K = 20) are compiled into every source file.
- Reported timing covers only the similarity and prediction phases: the `[Timing] Total (sim+pred)` line that each binary prints, excluding data generation and evaluation.

Measurement protocol. Speedup is computed as $S(p) = T_{\text{serial}}/T_{\text{parallel}}(p)$ from the total similarity plus prediction time. Thread and process counts are swept over $\{1, 2, 4, 8\}$; both hybrid variants (MPI+OpenMP and MPI+Pthreads) are measured at the four $P \times T$ layouts (2×4 , 4×2 , 8×1 , 1×8 , all at eight total workers); and CUDA is benchmarked once on the GPU. Correctness is reported with MAE/RMSE against the serial prediction array and the [Check] similarity checksum emitted by every run.

4 Correctness Analysis: Parallel vs. Serial

For implementation correctness, the expected value is the serial prediction and the observed value is the prediction produced by each parallel implementation for the same (u, i) pair. This interpretation is used throughout the report:

$$\text{MAE} = \frac{1}{|T|} \sum_{(u,i) \in T} |\hat{r}_{ui}^{\text{par}} - \hat{r}_{ui}^{\text{ser}}|, \quad \text{RMSE} = \sqrt{\frac{1}{|T|} \sum_{(u,i) \in T} (\hat{r}_{ui}^{\text{par}} - \hat{r}_{ui}^{\text{ser}})^2}.$$

If every implementation reproduces the serial prediction array exactly, both metrics are 0.0000. The serial row is also 0.0000 because it is the baseline compared against itself.

Version	Count	MAE	RMSE	Checksum
Serial	1	0.0000	0.0000	942.387323
OpenMP	1T	0.0000	0.0000	942.387323
OpenMP	2T	0.0000	0.0000	942.387323
OpenMP	4T	0.0000	0.0000	942.387323
OpenMP	8T	0.0000	0.0000	942.387323
Pthreads	1T	0.0000	0.0000	942.387323
Pthreads	2T	0.0000	0.0000	942.387323
Pthreads	4T	0.0000	0.0000	942.387323
Pthreads	8T	0.0000	0.0000	942.387323
MPI	1P	0.0000	0.0000	942.387323
MPI	2P	0.0000	0.0000	942.387323
MPI	4P	0.0000	0.0000	942.387323
MPI	8P	0.0000	0.0000	942.387323
Hybrid 2×4	8	0.0000	0.0000	942.387323
Hybrid 4×2	8	0.0000	0.0000	942.387323
Hybrid 8×1	8	0.0000	0.0000	942.387323
Hybrid 1×8	8	0.0000	0.0000	942.387323
Hybrid PT 2×4	8	0.0000	0.0000	942.387323
Hybrid PT 4×2	8	0.0000	0.0000	942.387323
Hybrid PT 8×1	8	0.0000	0.0000	942.387323
Hybrid PT 1×8	8	0.0000	0.0000	942.387323
CUDA	GPU	0.0000	0.0000	942.387323

Table 3: Implementation correctness and bit-identical similarity checksum. All 22 runs (21 CPU + 1 GPU) produce MAE = 0.0000 and RMSE = 0.0000 against the serial prediction array.

As Table 3 shows, all 22 runs produce MAE = **0.0000**, RMSE = **0.0000**, and checksum = **942.387323**, confirming zero race conditions and correct partitioning across every implementation. Crucially, the CUDA version, despite using completely different floating point hardware and a register based TOP K selection rather than the CPU’s `qsort`, reproduces the serial result exactly. The MPI+Pthreads hybrid also matches exactly, confirming that swapping the intra

node threading model from OpenMP to raw pthreads is purely a *runtime* change and does not perturb the algorithm or partitioning.

5 Timing Measurements and Performance Analysis

5.1 Performance Metric Definitions

$$S(p) = \frac{T_{\text{serial}}}{T_{\text{parallel}}(p)} \quad E(p) = \frac{S(p)}{p} \quad S_{\text{max}}(p) = \frac{1}{f + (1-f)/p}$$

The measured speedups are interpreted with Amdahl’s Law as an upper bound model, not as a precise fitted decomposition. The benchmark total used below is `similarity + prediction`, so setup, data generation, and evaluation are outside the reported speedup denominator. The gap between the ideal eight worker limit and the measured CPU results is therefore mainly explained by practical overheads: scheduling, communication, memory bandwidth, and reduced all core turbo clocks.

5.2 Serial Baseline

Phase	Time (s)	% of Total
Similarity matrix (Phase 3)	1.3239	15.5%
Prediction phase (Phase 4)	7.2152	84.5%
Total (sim + pred)	8.5390	100%

Table 4: Serial baseline timing ($N = M = 1,000$). Prediction dominates ($\approx 85\%$), so Phase 4 is the larger speedup opportunity.

5.3 OpenMP: Varying Thread Count

Threads	Sim (s)	Pred (s)	Total (s)	Speedup	Efficiency
1	1.5565	7.0694	8.6259	0.99	0.99
2	0.8081	3.6979	4.5060	1.90	0.95
4	0.4205	1.9787	2.3992	3.56	0.89
8	0.2390	1.1251	1.3642	6.26	0.78

Table 5: OpenMP results. Speedup relative to serial baseline (8.5390 s).

5.4 Pthreads: Varying Thread Count

Threads	Sim (s)	Pred (s)	Total (s)	Speedup	Efficiency
1	1.3332	7.2654	8.5986	0.99	0.99
2	0.9871	3.6410	4.6281	1.85	0.92
4	0.5892	2.1080	2.6972	3.17	0.79
8	0.3422	1.1446	1.4868	5.74	0.72

Table 6: Pthreads results.

5.5 MPI: Varying Process Count

Processes	Sim (s)	Pred (s)	Total (s)	Speedup	Efficiency
1	3.2286	7.4695	10.6981	0.80	0.80
2	1.6226	3.7774	5.4000	1.58	0.79
4	0.8644	2.0129	2.8774	2.97	0.74
8	0.4944	1.1815	1.6759	5.10	0.64

Table 7: MPI results. MPI 1P (10.70 s) > serial (8.54 s) due to process startup and MPI_Allgather overhead at $P = 1$, which is expected rather than a bug.

5.6 Hybrid MPI+OpenMP: Fixed 8 Total Workers

Config	P	T	Sim (s)	Pred (s)	Total (s)	S	E
Hybrid 2×4	2	4	1.2220	2.7739	3.9960	2.14	0.27
Hybrid 4×2	4	2	0.5627	1.2004	1.7631	4.84	0.61
Hybrid 8×1	8	1	0.5829	1.2140	1.7970	4.75	0.59
Hybrid 1×8	1	8	2.4116	5.3890	7.8006	1.09	0.14

Table 8: Hybrid MPI+OpenMP results (8 total workers). Best config is Hybrid 4×2 (4 ranks × 2 OMP threads) by a small margin over 8×1.

5.7 Hybrid MPI+Pthreads: Fixed 8 Total Workers

Config	P	T	Sim (s)	Pred (s)	Total (s)	S	E
Hybrid PT 2×4	2	4	1.1844	2.8401	4.0245	2.12	0.27
Hybrid PT 4×2	4	2	0.5840	1.1967	1.7807	4.80	0.60
Hybrid PT 8×1	8	1	0.5048	1.2654	1.7702	4.82	0.60
Hybrid PT 1×8	1	8	2.4055	5.5972	8.0027	1.07	0.13

Table 9: Hybrid MPI+Pthreads results (8 total workers). Same shape as the MPI+OpenMP hybrid: best config is 8×1 (essentially pure MPI) ahead of 4×2 by less than 1%, both at ≈ 0.6 efficiency; worst is 1×8, which pays MPI overhead with no distribution benefit.

5.8 CUDA: GPU Benchmark

The CUDA version was benchmarked on the same machine’s **NVIDIA GeForce RTX 3050 Laptop GPU** (Ampere, compute capability 8.6, 16 SMs, 4 GB), compiled with `nvcc -O2 -arch=sm_86`. Speedup is reported against the serial baseline ($T_{\text{serial}} = 8.5390$ s).

Phase / Metric	Time (s)	Notes
H→D transfer (ratings)	0.0009	PCIe, one time
Similarity matrix (Phase 3)	0.0863	GPU kernel (CUDA events)
Prediction phase (Phase 4)	0.1339	GPU kernel (CUDA events)
Total (sim + pred)	0.2202	kernel only
Speedup vs. serial	38.78×	vs. serial 8.5390 s

Table 10: CUDA timing on the RTX 3050 ($N = M = 1,000$). The similarity kernel launches $63 \times 63 \times 256 = 1,016,064$ threads; kernel time is isolated from host $\leftrightarrow$ device transfers via `cudaEventRecord`.

The GPU completes the combined similarity + prediction work in **0.22 s**, roughly **38.78×** faster than the serial baseline and about **6×** faster than the best 8 thread CPU configuration, because the embarrassingly parallel (u, v) and (u, i) workloads map directly onto the GPU’s thousands of cores. Host $\leftrightarrow$ device transfer cost is negligible (below one millisecond) since the ratings matrix stays resident on the device across all three kernels.

5.9 Scalability Across Problem Sizes

To check how each model holds up as the workload grows, the three fastest configurations (Serial baseline, OpenMP 8T, MPI 8P, and CUDA) are run again at 500×500 , $1,000 \times 1,000$, and $2,000 \times 2,000$. Table 11 reports the observed times and speedups (from `results/scaling_summary.csv`); note that the serial time at $N = M = 1,000$ here (8.7561 s) is a separate run from the one used as the strong scaling baseline in §5.2 (8.5390 s); the two differ by $\approx 2.5\%$, typical run to run noise.

Size	Serial (s)	OMP 8T (s)	Speedup	MPI 8P (s)	Speedup	CUDA (s)	Speedup
500×500	0.9883	0.1531	6.45×	0.2535	3.89×	0.0303	32.61×
1000×1000	8.7561	1.3980	6.26×	1.6814	5.20×	0.2206	39.69×
2000×2000	78.6369	12.9864	6.05×	16.1278	4.87×	1.4227	55.27×

Table 11: Problem size sweep at the fastest worker count per technology. Serial grows as expected ($O(N^2M)$: $0.99 \rightarrow 8.76 \rightarrow 78.64$ s tracks the N^3 growth factor). OpenMP speedup is stable ($6.45 \rightarrow 6.26 \rightarrow 6.05$); MPI improves from 500×500 (where `MPI_Allgatherv` is not yet amortised) before declining at the largest size; **CUDA speedup grows with N from $32.6\times$ to $55.3\times$** as the GPU fills its SMs and amortises launch and PCIe overheads.

What the trend reveals. The three models exhibit qualitatively different scaling behaviour. *OpenMP* hovers within 6% of the same speedup across the three sizes, reflecting a model that is essentially compute bound on this CPU at any size ≥ 500 . *MPI* is communication sensitive: at 500×500 the ≈ 1 MB `MPI_Allgatherv` dominates the small compute window (only $3.89\times$); at $1,000$ it amortises ($5.20\times$); at $2,000$ the rebuilt similarity matrix swells to ≈ 16 MB per rank and efficiency falls back ($4.87\times$). *CUDA* exhibits the opposite of MPI: the GPU has spare SMs at the smallest size and its speedup climbs sharply with problem size, hitting $55.3\times$ at $2,000 \times 2,000$; the model that already wins at small N widens its lead as the data grows.

6 Performance Discussion

6.1 Head to Head Comparison at 8 Workers

Version	Total (s)	Speedup	Efficiency	MAE
Serial (baseline)	8.5390	1.00	1.00	0.0000
CUDA (GPU)	0.2202	38.78	n/a	0.0000
OpenMP 8T	1.3642	6.26	0.78	0.0000
Pthreads 8T	1.4868	5.74	0.72	0.0000
MPI 8P	1.6759	5.10	0.64	0.0000
Hybrid 4×2	1.7631	4.84	0.61	0.0000
Hybrid PT 8×1	1.7702	4.82	0.60	0.0000
Hybrid PT 4×2	1.7807	4.80	0.60	0.0000
Hybrid 8×1	1.7970	4.75	0.59	0.0000
Hybrid 2×4	3.9960	2.14	0.27	0.0000
Hybrid PT 2×4	4.0245	2.12	0.27	0.0000
Hybrid 1×8	7.8006	1.09	0.14	0.0000
Hybrid PT 1×8	8.0027	1.07	0.13	0.0000

Table 12: Head to head: CUDA (GPU) plus all CPU versions at 8 workers. CUDA is the fastest overall; OpenMP 8T is the best CPU configuration. The MAE column shows implementation correctness against the serial prediction array, identical at 0.0000 for every row. (GPU efficiency per core is not directly comparable, so omitted.)

6.2 OpenMP vs. Pthreads

OpenMP ($\times 6.26$, 78% efficiency) outperforms Pthreads ($\times 5.74$, 72%) for three reasons. First, *dynamic load balancing*: `schedule(dynamic,4)` adapts to the triangular loop imbalance, which the static ceiling division in Pthreads cannot. Second, *thread reuse*: OpenMP reuses one thread team across Phases 2–4, whereas Pthreads creates and joins threads three times per run. Third, *runtime optimisations*: GCC’s OpenMP runtime applies platform specific scheduling heuristics. The two are closest at low thread counts (within 1% at $T = 1$) and the gap widens with thread count, as expected when the triangular imbalance has more workers to redistribute.

6.3 MPI Scaling

MPI efficiency declines from 80% ($P = 1$) to 64% ($P = 8$). The `MPI_Allgatherv` that rebuilds the $N \times N$ similarity matrix (≈ 4 MB) limits single node performance. These results do not measure multi node execution; they show that MPI is useful but carries communication cost on the single node platform used here.

6.4 Hybrid MPI+OpenMP: Inversion on a Single Node

On a single node, hybrid performance rises sharply with the MPI rank count and then flattens once the rank count matches the number of physical cores:

$$\text{Hybrid } 4 \times 2 (\times 4.84) \approx 8 \times 1 (\times 4.75) \gg 2 \times 4 (\times 2.14) > 1 \times 8 (\times 1.09).$$

The 4×2 and 8×1 configurations are statistically tied at $\approx 60\%$ efficiency. In this run 4×2 edges 8×1 by under 2%, having more aggressive OMP scheduling absorb the triangular imbalance within each rank’s slice. Below that, the picture is unambiguous: when fewer MPI ranks are used, the MPI startup and `MPI_Allgatherv` cost dominates what is essentially an OpenMP workload. The Hybrid 1×8 configuration pays all the MPI overhead while gaining zero distribution benefit. **On this single node setup, the hybrid model does not outperform pure OpenMP;**

its main justification is compatibility with systems where work must be split across MPI ranks.

6.5 Hybrid MPI+OpenMP vs. MPI+Pthreads

Comparing the two hybrid variants at matched $P \times T$ layouts isolates the effect of the intra node threading model: the outer MPI structure, the row partition, and the collective calls are identical in both. The MPI+OpenMP variant is faster wherever the inner team has more than one thread (4×2 : 1.7631 s vs. 1.7807 s; 2×4 : 3.9960 s vs. 4.0245 s; 1×8 : 7.8006 s vs. 8.0027 s), mirroring what is already seen at the single technology level between pure OpenMP ($6.26\times$) and pure Pthreads ($5.74\times$). The gap comes from the same two mechanisms: (i) OpenMP's `schedule(dynamic,4)` smooths the triangular Phase 3 imbalance that the pthreads `work_range` static partition cannot, and (ii) OpenMP reuses one thread team across phases, whereas the pthreads harness pays `pthread_create`/`pthread_join` overhead three times per run. At $T=1$, where the threading model is effectively bypassed and only the MPI layer matters, the two hybrids converge. In fact, Hybrid PT 8×1 (1.7702 s) edges Hybrid 8×1 (1.7970 s) by $\approx 1.5\%$, well within run to run noise, because there is no OpenMP runtime to amortise and the pthreads binary skips the OMP team creation cost entirely.

6.6 Amdahl's Law

Amdahl's Law explains why the CPU implementations remain below the ideal eight worker limit. The similarity and prediction phases are highly parallel, but the measured speedups also include scheduling overhead, memory bandwidth pressure, MPI collectives, and lower all core turbo frequency. OpenMP therefore comes closest to the ideal, Pthreads loses additional efficiency through static work splitting and thread creation, and MPI loses more to collective communication.

7 Conclusions

1. **All versions preserve correctness.** MAE and RMSE are computed against the serial prediction array. Every implementation achieves MAE = **0.0000** and RMSE = **0.0000**, and the similarity-matrix checksum is bit-identical at **942.387323**.
2. **OpenMP achieves the best CPU performance.** $\times 6.26$ speedup at 8 threads, 78% efficiency. Dynamic scheduling and thread reuse explain the advantage.
3. **Pthreads is competitive but less efficient.** $\times 5.74$ at 8T, 72% efficiency. Static row assignment and per phase thread creation limit scalability.
4. **MPI achieves good speedup with communication overhead.** $\times 5.10$ at 8P, 64% efficiency. On this single node benchmark, MPI_Allgather communication keeps it behind OpenMP.
5. **Hybrid performance is best when the inner thread count is small on a single node.** The four MPI+OpenMP configurations rank 4×2 ($\times 4.84$) $\gtrsim$ 8×1 ($\times 4.75$) $\gg$ 2×4 ($\times 2.14$) $>$ 1×8 ($\times 1.09$); the MPI+Pthreads variant ranks 8×1 ($\times 4.82$) $\gtrsim$ 4×2 ($\times 4.80$) $\gg$ 2×4 ($\times 2.12$) $>$ 1×8 ($\times 1.07$). The two hybrids agree to within $\approx 2\%$ at every layout. The OMP variant wins at $T > 1$ (dynamic scheduling), while the PT variant wins at $T = 1$ (no OMP runtime overhead). On this single node run, both hybrids stay behind pure OpenMP.
6. **CUDA provides the largest speedup by far.** Benchmarked on the NVIDIA RTX 3050, the GPU completes similarity + prediction in **0.2202 s**, a **38.78 $\times$** speedup over the serial baseline and $\approx 6\times$ faster than the best 8 thread CPU run. The similarity kernel launches a grid with 1,016,064 logical threads.

Final ranking (single node):

CUDA GPU ($\times 38.78$) > OpenMP 8T ($\times 6.26$) > Pthreads 8T ($\times 5.74$) > MPI 8P ($\times 5.10$)
> Hybrid 4 $\times$ 2 ($\times 4.84$) > Hybrid PT 8 $\times$ 1 ($\times 4.82$) > Hybrid PT 4 $\times$ 2 ($\times 4.80$) > Hybrid 8 $\times$ 1
($\times 4.75$)
> Hybrid 2 $\times$ 4 ($\times 2.14$) > Hybrid PT 2 $\times$ 4 ($\times 2.12$) > Hybrid 1 $\times$ 8 ($\times 1.09$) > Hybrid PT 1 $\times$ 8
($\times 1.07$)

References

1. J. S. Breese, D. Heckerman, and C. Kadie, “Empirical analysis of predictive algorithms for collaborative filtering,” *arXiv:1301.7363*, 2013.
2. B. Sarwar, G. Karypis, J. Konstan, and J. Riedl, “Item based collaborative filtering recommendation algorithms,” in *Proc. WWW*, pp. 285–295, 2001.
3. B. Chapman, G. Jost, R. van der Pas, *Using OpenMP*. MIT Press, 2008.
4. W. Gropp, E. Lusk, A. Skjellum, *Using MPI*. MIT Press, 1994.
5. J. Nickolls et al., “Scalable parallel programming with CUDA,” *Queue*, vol. 6, no. 2, pp. 40–53, 2008.